목차

- 이미지 분류
- 파인튜닝

🎨 1. 이미지 분류 (Image Classification)

```
from transformers import pipeline
from PIL import Image
import requests

# 이미지 분류 파이프라인 생성
classifier = pipeline("image-classification")

# 방법 1: 인터넷 이미지 URL 사용
img_url = "https://huggingface.co/datasets/huggingface/documentation-images/resolve/main/beignets-task-guide.png"
image = Image.open(requests.get(img_url, stream=True).raw)

# 이미지 분류 실행
results = classifier(image)

print("🖼️ 이미지 분류 결과:")
for result in results[:3]:  # 상위 3개 결과만
    print(f"  • {result['label']}: {result['score']:.2%}")

# 방법 2: 로컬 이미지 파일 사용
# image = Image.open("my_photo.jpg")
# results = classifier(image)
```

```
No model was supplied, defaulted to google/vit-base-patch16-224 and revision 3f49326 (https://huggingface.co/google/vit-base-patch16-224).
Using a pipeline without specifying a model name and revision in production is not recommended.
config.json:         69.7k/? [00:00<00:00, 1.32MB/s]
model.safetensors: 100%                                                  346M/346M [00:09<00:00, 58.7MB/s]
preprocessor_config.json: 100%                                           160/160 [00:00<00:00, 5.83kB/s]
Fast image processor class <class 'transformers.models.vit.image_processing_vit_fast.ViTImageProcessorFast'> is available for this model. U
Device set to use cpu
🖼️ 이미지 분류 결과:
  • espresso: 18.18%
  • eggnog: 12.61%
  • coffee mug: 10.04%
```

2. 파인튜닝

```
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import Dataset

# 1️⃣ 모델 로드
model = AutoModelForSequenceClassification.from_pretrained("beomi/kcbert-base", num_labels=2)
tokenizer = AutoTokenizer.from_pretrained("beomi/kcbert-base")

# 2️⃣ 데이터 준비
data = {
    "text": ["정말 좋아요!", "최악이에요", "추천합니다", "별로예요"],
    "label": [1, 0, 1, 0]
}
dataset = Dataset.from_dict(data)

# 3️⃣ 전처리
def preprocess(examples):
    return tokenizer(examples["text"], truncation=True, padding=True)

dataset = dataset.map(preprocess, batched=True)

# 4️⃣ 학습 설정
args = TrainingArguments(
    output_dir="./model",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    report_to="none"              # wandb 사용 안함.
)

# 5️⃣ 학습
```

```
trainer = Trainer(model=model, args=args, train_dataset=dataset)
trainer.train()

# ⭐ 6️⃣ 모델 제대로 저장하기 (이게 핵심!)
model.save_pretrained("./my_model")
tokenizer.save_pretrained("./my_model")

# 7️⃣ 사용하기
from transformers import pipeline
classifier = pipeline("text-classification", model="./my_model")
print(classifier("이 제품 마음에 들어요"))
```

```
Some weights of BertForSequenceClassification were not initialized from the model checkpoint at beomi/kcbert-base and are newly initialized
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
Map: 100%                                                 4/4 [00:00<00:00, 103.84 examples/s]
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:666: UserWarning: 'pin_memory' argument is set as true but no accele
  warnings.warn(warn_msg)
[6/6 00:16, Epoch 3/3]
 Step  Training Loss
Device set to use cpu
[{'label': 'LABEL_0', 'score': 0.5803873538970947}]
```

실전 예제: 고객 리뷰 감정 분석 파인튜닝

```
# ============================================================
# 실전 예제: 고객 리뷰 감정 분석 파인튜닝
# ============================================================

!pip install -q transformers datasets torch evaluate

from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer, TrainingArguments
from datasets import Dataset
import numpy as np
import evaluate

print("🎯 고객 리뷰 감정 분석 파인튜닝 시작!Wn")

# ============================================================
# 1단계: 데이터 준비
# ============================================================
print("📊 학습 데이터 준비 중...Wn")

# 실제 상황을 가정한 고객 리뷰 데이터
reviews = [
    # 긍정 리뷰 (label=2)
    "이 제품 정말 최고예요! 배송도 빠르고 품질도 좋아요.",
    "완전 만족합니다. 재구매 의사 100%!",
    "기대 이상이에요. 가격 대비 성능이 뛰어나네요.",
    "친구에게 추천하고 싶을 정도로 좋습니다.",
    "디자인도 예쁘고 실용적이에요. 강추!",

    # 중립 리뷰 (label=1)
    "그냥 그래요. 가격만큼은 하는 것 같아요.",
    "나쁘지 않은데 특별히 좋지도 않네요.",
    "평범합니다. 쓸만해요.",
    "기대했던 것과 비슷해요.",
    "그저 그런 제품이에요.",

    # 부정 리뷰 (label=0)
    "완전 실망했어요. 품질이 너무 안 좋습니다.",
    "배송도 늦고 제품도 불량이네요. 환불하고 싶어요.",
    "사진과 다르고 가격에 비해 형편없어요.",
    "별로예요. 돈 아깝습니다.",
    "두 번 다시 안 사고 싶네요. 최악입니다.",
]

labels = [2,2,2,2,2, 1,1,1,1,1, 0,0,0,0,0]  # 2=긍정, 1=중립, 0=부정

# Dataset 생성
train_data = Dataset.from_dict({"text": reviews, "label": labels})
print(f"✅ 학습 데이터: {len(reviews)}개 (긍정/중립/부정 각 5개)Wn")
```

```
━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━━ 84.1/84.1 kB 3.1 MB/s eta 0:00:00
🎯 고객 리뷰 감정 분석 파인튜닝 시작!

📊 학습 데이터 준비 중...
```

✅ 학습 데이터: 15개 (긍정/중립/부정 각 5개)

```
# ============================================================
# 2단계: 모델 및 토크나이저 로딩
# ============================================================
print("🤖 모델 로딩 중...\n")

model_name = "distilbert-base-uncased"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=3  # 긍정/중립/부정 3개 클래스
)

print("✅ 모델 로딩 완료!\n")
```

```
🤖 모델 로딩 중...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
tokenizer_config.json: 100%                 48.0/48.0 [00:00<00:00, 1.95kB/s]
config.json: 100%                           483/483 [00:00<00:00, 18.7kB/s]
vocab.txt: 100%                             232k/232k [00:00<00:00, 5.45MB/s]
tokenizer.json: 100%                        466k/466k [00:00<00:00, 9.78MB/s]
model.safetensors: 100%                     268M/268M [00:11<00:00, 22.6MB/s]
Some weights of DistilBertForSequenceClassification were not initialized from the model checkpoint at distilbert-base-uncased and are newly
You should probably TRAIN this model on a down-stream task to be able to use it for predictions and inference.
✅ 모델 로딩 완료!
```

```
# ============================================================
# 3단계: 데이터 전처리
# ============================================================
print("🔤 데이터 전처리 중...\n")

def tokenize(examples):
    return tokenizer(examples["text"], padding="max_length", truncation=True, max_length=128)

tokenized_data = train_data.map(tokenize, batched=True)
print("✅ 전처리 완료!\n")
```

```
🔤 데이터 전처리 중...

Map: 100%                                   15/15 [00:00<00:00, 268.05 examples/s]
✅ 전처리 완료!
```

```
# ============================================================
# 4단계: 평가 지표 설정
# ============================================================
metric = evaluate.load("accuracy")

def compute_metrics(eval_pred):
    logits, labels = eval_pred
    predictions = np.argmax(logits, axis=-1)
    return metric.compute(predictions=predictions, references=labels)

# ============================================================
# 5단계: 학습 설정
# ============================================================
print("⚙️ 학습 파라미터 설정...\n")

training_args = TrainingArguments(
    output_dir="./review_sentiment_model",
    eval_strategy="epoch",
    learning_rate=2e-5,
    per_device_train_batch_size=4,
    num_train_epochs=5,
    weight_decay=0.01,
    logging_steps=5,
)
```

```
trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized_data,
    eval_dataset=tokenized_data,  # 실전에서는 별도 검증셋 사용
    compute_metrics=compute_metrics,
)
```

```
Downloading builder script:       4.20k/? [00:00<00:00, 152kB/s]
⚙️ 학습 파라미터 설정...
```

```
# ============================================================
# 6단계: 파인튜닝 실행
# ============================================================
print("🚀 파인튜닝 시작!\n")
print("="*60)

trainer.train()

print("="*60)
print("\n✅ 파인튜닝 완료!\n")
```

```
🚀 파인튜닝 시작!

============================================================
/usr/local/lib/python3.12/dist-packages/notebook/notebookapp.py:191: SyntaxWarning: invalid escape sequence '\/'
  | |_| | '_ \/ _` / _` |  _/ -_)
wandb: Logging into wandb.ai. (Learn how to deploy a W&B server locally: https://wandb.me/wandb-server)
wandb: You can find your API key in your browser here: https://wandb.ai/authorize?ref=models
wandb: Paste an API key from your profile and hit enter: · · · · · · · · · ·
wandb: WARNING If you're specifying your api key in code, ensure this code is not shared publicly.
wandb: WARNING Consider setting the WANDB_API_KEY environment variable, or running `wandb login` from the command line.
wandb: No netrc file found, creating one.
wandb: Appending key for api.wandb.ai to your netrc file: /root/.netrc
wandb: Currently logged in as: colabstart (colabstart-edu) to https://api.wandb.ai. Use `wandb login --relogin` to force relogin
Tracking run with wandb version 0.22.2
Run data is saved locally in /content/wandb/run-20251014_062716-mhqmfk66
Syncing run devoted-shape-2 to Weights & Biases (docs)
View project at https://wandb.ai/colabstart-edu/huggingface
View run at https://wandb.ai/colabstart-edu/huggingface/runs/mhqmfk66
[20/20 00:04, Epoch 5/5]
```

Epoch	Training Loss	Validation Loss	Accuracy
1	No log	1.095709	0.333333
2	1.133700	1.085720	0.333333
3	1.089700	1.079347	0.333333
4	1.106800	1.071627	0.333333
5	1.076100	1.066991	0.333333

```
============================================================

✅ 파인튜닝 완료!
```

```
# ============================================================
# 7단계: 학습된 모델 테스트 (GPU 버전 수정)
# ============================================================
print("🪄 실전 테스트\n")
print("="*60)

test_reviews = [
    "배송이 정말 빨라요! 제품도 마음에 들어요.",
    "그냥 평범한 제품이네요.",
    "돈만 날렸어요. 품질 최악입니다.",
    "가성비 좋아요. 추천합니다!",
    "기대에는 못 미치지만 쓸만해요.",
]

label_names = ["😞 부정", "😐 중립", "😊 긍정"]

# 평가 모드로 전환 (Dropout 등 비활성화)
model.eval()

for review in test_reviews:
    # 1. 토크나이징
    inputs = tokenizer(review, return_tensors="pt", padding=True, truncation=True)
```

```
    # 2. 입력 데이터를 모델과 같은 디바이스로 이동! (핵심 수정)
    inputs = {key: value.to(device) for key, value in inputs.items()}

    # 3. 추론 시 gradient 계산 비활성화 (속도 향상, 메모리 절약)
    with torch.no_grad():
        outputs = model(**inputs)

    # 4. 예측 결과 추출
    prediction = outputs.logits.argmax(-1).item()
    confidence = outputs.logits.softmax(-1).max().item()

    print(f"₩n리뷰: {review}")
    print(f"예측: {label_names[prediction]} (확신도: {confidence:.1%})")

print("₩n" + "="*60)
print("✅ 테스트 완료!")
print("="*60)
```

```
🪄 실전 테스트

============================================================

리뷰: 배송이 정말 빨라요! 제품도 마음에 들어요.
예측: 😞 부정 (확신도: 35.7%)

리뷰: 그냥 평범한 제품이네요.
예측: 😞 부정 (확신도: 36.3%)

리뷰: 돈만 날렸어요. 품질 최악입니다.
예측: 😞 부정 (확신도: 37.5%)

리뷰: 가성비 좋아요. 추천합니다!
예측: 😞 부정 (확신도: 35.9%)

리뷰: 기대에는 못 미치지만 쓸만해요.
예측: 😞 부정 (확신도: 36.1%)

============================================================
✅ 테스트 완료!
============================================================
```

실전 예제 : 제품 설명 자동 생성 파인튜닝

```
# ============================================================
# 실전 예제: 제품 설명 자동 생성 파인튜닝 (수정판)
# ============================================================
!pip install -q transformers datasets torch sentencepiece

# 변경 (Seq2Seq 전용 아규먼트 + 데이터 콜레이터 포함)
from transformers import (
    AutoTokenizer,
    AutoModelForSeq2SeqLM,
    Trainer,
    Seq2SeqTrainingArguments,
    DataCollatorForSeq2Seq,
    EarlyStoppingCallback,
)


from datasets import Dataset, DatasetDict
import numpy as np, torch

print("🎯 제품 설명 자동 생성 파인튜닝 시작!₩n")

# 1) 데이터 ------------------------------------------------
print("📊 학습 데이터 준비 중...₩n")
product_data = {
    "input": [
        "무선 이어폰, 블루투스 5.0, 노이즈 캔슬링, 30시간 재생",
        "스마트워치, 심박수 측정, 방수, 7일 배터리",
        "노트북, 15.6인치, 16GB RAM, 512GB SSD, 가벼움",
        "전기면도기, 3중날, 방수, USB 충전",
        "공기청정기, HEPA 필터, 3단계, 조용함",
    ],
    "output": [
        "프리미엄 무선 이어폰으로 탁월한 노이즈 캔슬링 기능을 제공합니다. 블루투스 5.0으로 안정적인 연결이 가능하며, 한 번 충전으로 최대 3
        "건강을 한눈에! 실시간 심박수 측정과 방수 기능을 갖춘 스마트워치입니다. 7일 동안 사용 가능한 강력한 배터리로 매일 건강을 체크하세요
        "업무와 엔터테인먼트를 한번에! 15.6인치 대화면에 16GB RAM과 512GB SSD로 빠른 성능을 자랑합니다. 휴대가 간편한 경량 디자인으로 어디
        "편안한 면도의 완성! 3중날 시스템으로 깔끔한 면도를 제공하며, 방수 기능으로 샤워 중에도 사용 가능합니다. USB 충전으로 언제 어디서나
        "깨끗한 공기로 건강한 생활! HEPA 필터가 미세먼지를 99.97% 제거하며, 3단계 조절 기능과 조용한 작동으로 숙면을 방해하지 않습니다.",
    ]
```

```
}

full = Dataset.from_dict(product_data)
# 검증을 최소로라도 분리 (작은 데이터지만 모니터링용)
ds = DatasetDict({
    "train": full.select(range(4)),
    "validation": full.select(range(4,5))
})
print(f"✅ 학습 데이터: {len(ds['train'])}개, 검증: {len(ds['validation'])}개₩n")
```

```
🎯 제품 설명 자동 생성 파인튜닝 시작!

📊 학습 데이터 준비 중...

✅ 학습 데이터: 4개, 검증: 1개
```

```
# 2) 모델/토크나이저 ----------------------------------------
print("🤖 한국어 T5 모델 로딩 중...₩n")
model_name = "paust/pko-t5-base"  # ← 한국어 T5 권장
tokenizer = AutoTokenizer.from_pretrained(model_name, use_fast=True)
model = AutoModelForSeq2SeqLM.from_pretrained(model_name)
print("✅ 모델 로딩 완료!₩n")

# 3) 전처리 ------------------------------------------------
print("🔤 데이터 전처리 중...₩n")
PREFIX = "제품 설명 생성(문장 2~3개, '~습니다' 톤, 과장 금지, 핵심 혜택 먼저): "

def preprocess(examples):
    inputs = [PREFIX + x for x in examples["input"]]
    model_inputs = tokenizer(inputs, max_length=128, truncation=True, padding="max_length")
    # 최신 권장: 타깃은 text_target= 사용
    labels = tokenizer(text_target=examples["output"], max_length=256, truncation=True, padding="max_length")
    model_inputs["labels"] = labels["input_ids"]
    return model_inputs

tokenized = ds.map(preprocess, batched=True, remove_columns=ds["train"].column_names)
# 라벨 패딩 -100 처리: DataCollatorForSeq2Seq에서 자동 처리되도록 설정
data_collator = DataCollatorForSeq2Seq(tokenizer=tokenizer, model=model, label_pad_token_id=-100)
print("✅ 전처리 완료!₩n")
```

```
🤖 한국어 T5 모델 로딩 중...

/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens), set it as secret i
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
✅ 모델 로딩 완료!

🔤 데이터 전처리 중...

Map: 100%                                                  4/4 [00:00<00:00, 61.75 examples/s]

Map: 100%                                                  1/1 [00:00<00:00, 25.92 examples/s]

✅ 전처리 완료!
```

```
# 4) 학습 설정 ----------------------------------------------
print("⚙️ 학습 파라미터 설정...₩n")
import inspect
from transformers import Seq2SeqTrainingArguments

# 1) 시그니처에서 어떤 키를 받는지 확인
sig = inspect.signature(Seq2SeqTrainingArguments.__init__)
USE_EVAL_KEY = "evaluation_strategy" if "evaluation_strategy" in sig.parameters else "eval_strategy"

# 2) 공통 인자 사전
common_kwargs = dict(
    output_dir="./product_description_model",
    num_train_epochs=3,
    per_device_train_batch_size=2,
    learning_rate=5e-5,
    logging_steps=1,
    save_total_limit=2,
    predict_with_generate=True,
    generation_max_length=120,
    generation_num_beams=6,
    label_smoothing_factor=0.1,
    load_best_model_at_end=True,
    metric_for_best_model="eval_loss",
```

```
    greater_is_better=False,
    report_to="none",
    seed=42,
)

# 3) 평가/저장 전략을 'epoch'로 통일 (load_best_model_at_end 호환)
common_kwargs[USE_EVAL_KEY] = "epoch"   # ← 여기만 버전에 따라 키가 달라짐
common_kwargs["save_strategy"] = "epoch"

training_args = Seq2SeqTrainingArguments(**common_kwargs)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=tokenized["train"],
    eval_dataset=tokenized["validation"],
    data_collator=data_collator,
    tokenizer=tokenizer,
    callbacks=[EarlyStoppingCallback(early_stopping_patience=2)],
)

# 5) 학습 ------------------------------------------------------
print("🚀 파인튜닝 시작!Wn")
print("="*60)
trainer.train()
print("="*60)
print("Wn✅ 파인튜닝 완료!Wn")
```

```
⚙️ 학습 파라미터 설정...

/tmp/ipython-input-3571869193.py:35: FutureWarning: `tokenizer` is deprecated and will be removed in version 5.0.0 for `Trainer.__init__`.
  trainer = Trainer(
🚀 파인튜닝 시작!

============================================================
[6/6 02:43, Epoch 3/3]
```

Epoch	Training Loss	Validation Loss
1	22.376500	18.788593
2	18.696700	17.839642
3	17.655700	17.675079

```
There were missing keys in the checkpoint model loaded: ['encoder.embed_tokens.weight', 'decoder.embed_tokens.weight'].
============================================================

✅ 파인튜닝 완료!
```

```
# 6) 테스트 ------------------------------------------------------
import torch
print("🧪 실전 테스트Wn")
print("="*60)

test_products = [
    "블루투스 스피커, 방수, 10시간 재생, 저음 강화",
    "커피머신, 원터치, 15바 압력, 자동 세척",
    "게이밍 마우스, RGB, 16000 DPI, 프로그래밍 가능",
]

model.eval()
for product in test_products:
    prompt = PREFIX + product  # 예: "제품 설명 생성(문장 2~3개, 과장 금지): "
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)

    with torch.inference_mode():
        output_ids = model.generate(
            **inputs,
            max_new_tokens=60,
            min_new_tokens=25,            # 너무 짧게 끝나는 것 방지
            do_sample=False,              # 결정론(재현성)
            num_beams=6,                  # 빔↑ → 품질↑(느려짐)
            length_penalty=1.4,           # 너무 길거나 짧은 문장 방지
            no_repeat_ngram_size=3,       # 반복 억제
            repetition_penalty=1.3,       # 단어 재탕 패널티
            encoder_no_repeat_ngram_size=3,
            eos_token_id=tokenizer.eos_token_id,
            pad_token_id=tokenizer.pad_token_id,
            renormalize_logits=True
        )

    text = tokenizer.decode(output_ids[0], skip_special_tokens=True)
```

```
    # 프리픽스 제거(가독성)
    desc = text.replace(prompt, "").strip()

    print(f"₩n제품 속성: {product}")
    print(f"생성된 설명:₩n{desc}")
    print("-"*60)

print("₩n" + "="*60)
print("✅ 테스트 완료!")
print("="*60)
```

```
💉 실전 테스트

============================================================

제품 속성: 블루투스 스피커, 방수, 10시간 재생, 저음 강화
생성된 설명:
문장 제품 소개 생성, 블루투스,, 고음 강화, 우퍼 강화, 스피커 설명 제품제품
------------------------------------------------------------

제품 속성: 커피머신, 원터치, 15바 압력, 자동 세척
생성된 설명:
과장 제품 소개 생성, 문장 1~2개,커피머신, 스테인리스: 자동, 3단계 압력, 커피 추출, 원두 분쇄
------------------------------------------------------------

제품 속성: 게이밍 마우스, RGB, 16000 DPI, 프로그래밍 가능
생성된 설명:
문장 제품 소개 생성, 문장 1~2개,(문장)(문장한, 게이밍(문장 제품
------------------------------------------------------------

============================================================
✅ 테스트 완료!
============================================================
```

코딩을 시작하거나 AI로 코드를 생성하세요.